ANSIBLE TEST INFOBIP: DESIGN DOCUMENT

By: Mateo Penayo

Introduction

I was given a task which included creating a playbook for a hypothetical ansible web server setup and configuration. In the following document I outline my thought process, the project which I decided to build and why I made certain decisions. A Github repository is also included in which you may be able to see the commit history and the changes that were made as the project advanced. In this document, however, I only explain the final state of the project. I will also explain when and where I used A.I. during development.

Before moving on, it is important to note that I decided to build a whole deployable project, including an inventory, templates, group_vars configuration file and some capabilities not required such as webserver configuration and blocks to handle errors. I decided to add these features in order to show my capabilities in working with them, as well as creating a Github repository with the same purpose, and to show the commit history.

Overall project structure

The directories and files of this project are organized as follows:

AnsibleTest/

├── ansible.cfg # points inventory at ./inventory.yaml

├── inventory.yaml # hosts in the `webservers` group

├── install_setup_webserver.yaml # the only playbook

├── README.md # one-line repo description

├── README_group_vars.md # detailed group_vars reference

├── README_DESIGN.md # this file

├── group_vars/

│ └── webservers.yml # ALL configuration lives here

└── templates/

```
├── index.html.j2        # shared default index page (both backends)
├── nginx.conf.j2        # nginx global config
├── server_block.conf.j2  # nginx per-site config (looped)
├── httpd.conf.j2        # httpd global config
└── vhost.conf.j2        # httpd per-site config (looped)
```

Scope

This project deploys a webserver, does an initial sample configuration, ensures that the webserver is running and checks for failures. It is intended to be used with either nginx or httpd webservers running on Debian or RHEL target OS. I could test directly on RHEL, but that would be the next step in this project.

Design decisions

- Single switch variable: webserver_package is the single source of truth in the group_vars config. It must always be set to either nginx or httpd. It controls what package is installed on the target host managed node.
- No roles: I briefly considered adding roles to the project but decided against it for the time being as it would add another layer of complexity for what is a simple task in essence. This would also be expanded in a real enterprise scenario, particularly if another backend is added.
- Single playbook: Separating httpd/nginx installation into different playbooks could also be considered. This would be simpler and easier to debug. In a real-world scenario, I would choose this option, however, I decided to keep it as a single playbook, both because it is consistent with the deliverables asked and because it shows better templating and variables management from my part.
- Reverse Proxy + SSL configuration example: For the nginx configuration I decided to include a sample backend reverse proxy and SSL configuration. This is simply to show an example of how such configuration may work. I did not do the same for httpd as it would make the template significantly longer. Actual SSL configuration would involve more than simply providing a path to a certificate, it may involve using certbot.
- OS-family branching inside templates: Similarly to the single playbook, it may be a better idea to include different templates for different OS families and have them called separately. This would also be the next step in design.

My workflow

To work on this project, I started by reading the official documentation and watching tutorials to understand how ansible works.

Then I sketched a rough design of the tasks, files, and tools that I would use.

After that I created a sample barebones project to test basics, syntax, ad-hoc commands and targeting several machines with simple plays such as pings or printing messages.

After that I started working on the main project. That consisted of following this rough loop:

1. Create a new task or series of tasks to accomplish one of the goals (deploying a webserver, configuring it, updating it, etc.)
2. Creating the necessary templates and variables that I think the task should need.
3. Debugging.
4. Fine tuning and documentation.

Used tools

The tools that I used to accomplish this project in ansible include: configuration files, inventory files, variables, handlers, loops, conditionals, blocks and templating. I did not need to use any collections or plugins beside ansible.builtin for this project. I considered the use of roles, but decided against it.

Project flow

The tasks of this project's playbook roughly consist of the following:

1. Stop and disable the inactive backend if installed.
2. Install the selected backend's package.
3. Deploy index page.
4. Deploy backend-specific global config.
5. Remove the placegolder site.
6. For httpd, create document_root for each non-redirect vhost.
7. Deploy per-site configs in a loop.
8. Validate full config.
9. If validation failed, show a message and abort.
10. Start + enable the service.
11. Reload the service if any task called the handler.

How I used A.I.

I used A.I. on this project in three main categories:

1. Solving questions: When I started encountering errors, having syntax mistakes or not being sure about a certain behavior, I would ask the A.I. to read documentation or to point me to where I can find it.
2. Default configuration and templating: For standard, boilerplate configuration and templates A.I. can be a great tool. This is because these were examples of how to deploy a configuration and as such, they didn't really matter. Certain configurations are pretty much standard for any webserver and can be a bit finicky in their syntax.
3. Documentation and comments: After I solved a problem, created a new task, introduced new variables or made a significant change, asking the A.I. to properly document the changes saves a lot of time.

Problems I encountered and how I solved them

Apart from small problems, such as the occasional missed YAML indentation, forgetting a colon, or similar issues, here are some of the problems I encountered in this project and what I did to solve or avoid them:

- Path problem: both nginx and apache run are installed in /sbin/ instead of /bin/. So, I then added a task that added the /sbin/ directory to the system $PATH variable.
- Apache/httpd package name: When I tested with nginx, I had a single variable for package_name and package_service. This worked since nginx shares a package and service name. When I added the httpd capabilities, I had to split them in two, since httpd uses apache as a service name.
- Leftover inactive server port connections: After I tested with nginx, httpd would fail, since or viceversa. This is because the ports (80 and 443) were still bound to the last service. To avoid this problem, I created a task that stopped and disabled the inactive web server.
- Removing existing configuration files: After testing for failures, I would keep having problems even after fixing the configuration files. This is because even if the variables in group_vars were changed, the existing configuration files in the host machine were not. To solve this problem, I created a task that deleted all configuration files before execution. Note that this would not work if another system or user also had to manage configuration files, but if everything in this

host machine is configured through this ansible playbook, deleting and refreshing works fine.

- Defining variables based on system OS: I noticed that some of the documentation was RHEL or Debian exclusive. When creating an index path, I would have to consider what distribution I was working on to not reference nonexistent directories. Similar things would happen with nginx_user variable, package name for httpd, and others. In this case, using either Ansible facts or ansible_os_family helped e use a conditional to solve this problem.
- Ensuring vhost document roots exist: httpd requires a document_root for each vhost that is not a redirect. This was also solved using an httpd specific task.